TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

A LAB REPORT
ON

Machine Learning Pipeline: Generative vs. Discriminative Models, Model Capacity, and Cross-Entropy on the Iris Dataset

SUBMITTED BY:

Name: Nabina Thapa

Roll No.: 080BCT047

Lab No.: 08

SUBMITTED TO:

Department of Electronics
and Computer Engineering

Contents

1 Objectives

1. To build a complete **machine learning pipeline** on the Iris dataset, with stratified training, validation, and test splits.
2. To train and fairly compare a **generative model** (Gaussian Naive Bayes) and a **discriminative model** (Logistic Regression) on the same data.
3. To study **model capacity** using Decision Trees of varying depth, and to identify settings that underfit and settings that overfit.
4. To select a hyperparameter using validation data only, and to report an honest, single evaluation on the held-out test set.
5. To compute **cross-entropy** (log loss) for a probabilistic classifier and relate it to the ideas of **MLE and MAP** estimation.

2 Theory

A supervised learning pipeline consists of preparing data, choosing a model, training it, validating it, and finally evaluating it once on unseen data. The Iris dataset (150 samples, 4 numerical features, 3 classes) is small and clean, which makes it convenient for studying these steps directly rather than being distracted by data-cleaning issues.

2.1 Generative vs. Discriminative Models

A **generative** model, such as Gaussian Naive Bayes, learns the class-conditional distribution $p(x \mid y)$ for each class and uses Bayes' rule to obtain $p(y \mid x)$. A **discriminative** model, such as Logistic Regression, models $p(y \mid x)$ directly, without describing how the input itself was generated.

2.2 Model Capacity, Bias and Variance

Model capacity describes how flexible a model is. A model with too little capacity cannot represent the true decision boundary and **underfits** – it performs poorly on both the training and validation data. A model with too much capacity can memorize the training data, including its noise, and **overfits** – high training accuracy but a noticeably lower validation accuracy. **Bias** is the systematic error of an overly simple model (underfitting); **variance** is the sensitivity of a model to the particular training sample it saw (overfitting).

2.3 Cross-Entropy, MLE and MAP

For a predicted probability $\hat{p}_y$ assigned to the true class y, the cross-entropy (negative log-likelihood) of a single prediction is

$$\text{Cost} = -\log(\hat{p}_y).$$

As $\hat{p}_y \to 1$ the cost goes to 0; as $\hat{p}_y \to 0$ the cost diverges, so confidently wrong predictions are penalized heavily. **Maximum Likelihood Estimation (MLE)** chooses parameters θ that maximize the likelihood of the observed data, $\theta^{\text{MLE}} = \arg\max_\theta \prod_i p(x^{(i)} \mid \theta)$. **Maximum A Posteriori (MAP)** estimation additionally multiplies in a prior $p(\theta)$,

$\theta^{\text{MAP}} = \arg\max_\theta \prod_i p(x^{(i)} \mid \theta)\, p(\theta)$, which in Logistic Regression corresponds to L_2-regularizing the weights towards zero.

3 Methodology

The Iris dataset was loaded with `scikit-learn` and split exactly as specified in the lab sheet: an 80/20 stratified split sets aside the test set, and the remaining 80% is split 75/25 into training and validation sets. All four features were standardized with a `StandardScaler` fitted only on the training data. Python 3.12 with `scikit-learn` 1.9, `pandas` and `matplotlib` was used throughout; the code was written in a single script and run from the terminal.

4 Implementation

4.1 Data Preparation and Model Comparison

The dataset is split and scaled first, after which Gaussian Naive Bayes and Logistic Regression are trained on the same training data and scored on the validation set.

```
iris = load_iris()
X, y = iris.data, iris.target
classes = iris.target_names

X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42, stratify=
       y_temp
)

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_val = sc.transform(X_val)
X_test = sc.transform(X_test)

split_sizes = {"train": len(X_train), "val": len(X_val), "test"
   : len(X_test)}
print("split sizes ->", split_sizes)

# ----------------------------------------------------
# Part 2 - generative (Naive Bayes) vs discriminative (Logistic
    Regression)
# ----------------------------------------------------

nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
nb_val_acc = acc(nb_model, X_val, y_val)

```

```
logit_model = LogisticRegression(max_iter=1000, random_state=42)
logit_model.fit(X_train, y_train)
logit_val_acc = acc(logit_model, X_val, y_val)

print(f"NaiveBayes val acc = {nb_val_acc:.3f}, LogisticRegression val acc = {logit_val_acc:.3f}")

# horizontal bar this time just for a different look than a plain vertical bar
plt.figure(figsize=(5.5, 3))
plt.barh(["Logistic Regression\n(discriminative)", "Gaussian Naive Bayes\n(generative)"],
         [logit_val_acc, nb_val_acc], color=[TEAL, CORAL])
plt.xlim(0, 1)
plt.xlabel("validation accuracy")
plt.title("Model comparison on validation set")
for i, v in enumerate([logit_val_acc, nb_val_acc]):
    plt.text(v + 0.02, i, f"{v:.3f}", va="center")
plt.tight_layout()
plt.savefig(OUT_DIR + "gen_vs_disc.png")
plt.close()
```

Listing 1: `pipeline.py` – data split and model comparison

Command and output:

```
$ python pipeline.py
split sizes -> {'train': 90, 'val': 30, 'test': 30}
NaiveBayes val acc = 0.933, LogisticRegression val acc = 0.933
```

4.2 Decision Tree Capacity and Hyperparameter Selection

Decision Trees are trained at four depths, the best depth is chosen using validation accuracy, and that single choice is checked once on the test set.

```
depth_options = [1, 3, 5, None]
depth_labels = [str(d) for d in depth_options]
train_scores = []
val_scores = []

for depth in depth_options:
    clf = DecisionTreeClassifier(max_depth=depth, random_state=42)
    clf.fit(X_train, y_train)
    train_scores.append(acc(clf, X_train, y_train))
    val_scores.append(acc(clf, X_val, y_val))

```

```python
for d, tr, va in zip(depth_labels, train_scores, val_scores):
    print(f"depth={d:<4} train={tr:.3f}  val={va:.3f}")

# grouped bar chart instead of a line plot
x_pos = np.arange(len(depth_options))
width = 0.35
plt.figure(figsize=(6.5, 4))
plt.bar(x_pos - width / 2, train_scores, width, label="train", color=NAVY)
plt.bar(x_pos + width / 2, val_scores, width, label="validation", color=GOLD)
plt.xticks(x_pos, depth_labels)
plt.xlabel("max_depth")
plt.ylabel("accuracy")
plt.ylim(0, 1.1)
plt.title("Decision tree: train vs validation accuracy by depth")
plt.legend()
plt.tight_layout()
plt.savefig(OUT_DIR + "tree_capacity.png")
plt.close()

# ---------------------------------------------------
# Part 4 - pick max_depth using the validation set, then touch the test set once
# ---------------------------------------------------

best_pos = val_scores.index(max(val_scores))
chosen_depth = depth_options[best_pos]

best_tree = DecisionTreeClassifier(max_depth=chosen_depth, random_state=42)
best_tree.fit(X_train, y_train)
test_accuracy = acc(best_tree, X_test, y_test)

print("chosen depth:", chosen_depth, "-> test accuracy:", round(test_accuracy, 3))

# ---------------------------------------------------
# Part 5 - bias / variance talk (reusing numbers from part 3, nothing new to compute)
# ---------------------------------------------------

print("shallow tree (depth=1):", train_scores[0], val_scores[0])
print("unrestricted tree (depth=None):", train_scores[-1], val_scores[-1])
```

Listing 2: `pipeline.py` – capacity sweep and hyperparameter selection

Command and output:

```
$ python pipeline.py
depth=1    train=0.667  val=0.667
depth=3    train=0.989  val=0.900
depth=5    train=1.000  val=0.933
depth=None train=1.000  val=0.933
chosen depth: 5 -> test accuracy: 0.933
shallow tree (depth=1): 0.6666666666666666 0.6666666666666666
unrestricted tree (depth=None): 1.0 0.9333333333333333
```

4.3 Cross-Entropy and MLE/MAP Analysis

The validation log loss is computed for Logistic Regression, and the most confident correct and most confident wrong predictions are pulled out with `pandas` for closer inspection. Regularization strength C is then swept to illustrate the MLE/MAP trade-off.

```
proba = logit_model.predict_proba(X_val)
val_loss = log_loss(y_val, proba)
print("val log loss:", val_loss)

# build a little table with pandas, easier to slice/sort than
    doing it by hand
val_df = pd.DataFrame({
    "true_label": y_val,
    "pred_label": logit_model.predict(X_val),
})
val_df["true_class_name"] = val_df["true_label"].apply(lambda i
    : classes[i])
val_df["prob_of_true_class"] = [proba[i, y_val[i]] for i in
    range(len(y_val))]
val_df["loss"] = -np.log(val_df["prob_of_true_class"])
val_df["correct"] = val_df["true_label"] == val_df["pred_label"
    ]

most_confident_right = val_df[val_df["correct"]].sort_values("
    loss").iloc[0]
most_confident_wrong = val_df[~val_df["correct"]].sort_values("
    loss", ascending=False).iloc[0]

print("best correct prediction:\n", most_confident_right)
print("worst wrong prediction:\n", most_confident_wrong)

sorted_df = val_df.sort_values("loss").reset_index(drop=True)
bar_colors = [GREEN if c else CORAL for c in sorted_df["correct
    "]]

plt.figure(figsize=(6.5, 4))
plt.scatter(sorted_df.index, sorted_df["loss"], c=bar_colors, s
    =60, edgecolor="black", linewidth=0.4)
```

```
plt.axhline(val_loss, color=NAVY, linestyle=":", label=f"mean = {val_loss:.3f}")
plt.xlabel("validation sample (sorted by loss)")
plt.ylabel("cross entropy loss")
plt.title("Per-sample loss (green=correct, coral=wrong)")
plt.legend()
plt.tight_layout()
plt.savefig(OUT_DIR + "cross_entropy.png")
plt.close()

# ----------------------------------------------------
# Part 7 - MLE vs MAP, using regularization strength C as the knob
# ----------------------------------------------------

C_grid = np.logspace(-3, 3, 7)  # 0.001 ... 1000
norms = []
accs_for_C = []

for c in C_grid:
    m = LogisticRegression(C=c, max_iter=2000, random_state=42)
    m.fit(X_train, y_train)
    norms.append(np.linalg.norm(m.coef_))
    accs_for_C.append(acc(m, X_val, y_val))

fig, (top, bottom) = plt.subplots(2, 1, figsize=(6.5, 5.5), sharex=True)
top.plot(C_grid, norms, marker="o", color=TEAL)
top.set_xscale("log")
top.set_ylabel("||w||")
top.set_title("Weight norm vs regularization strength C")

bottom.plot(C_grid, accs_for_C, marker="s", color=CORAL)
bottom.set_xscale("log")
bottom.set_ylabel("val accuracy")
bottom.set_xlabel("C (log scale)")
bottom.set_ylim(0.5, 1.05)

fig.tight_layout()
fig.savefig(OUT_DIR + "mle_map.png")
plt.close(fig)
```

Listing 3: `pipeline.py` – cross-entropy and MLE/MAP sweep

Command and output:

```
$ python pipeline.py
val log loss: 0.21527889101774805
best correct prediction:
```

```
 true_label                  0
pred_label                   0
true_class_name         setosa
prob_of_true_class    0.996551
loss                  0.003455
correct                   True
Name: 12, dtype: object
worst wrong prediction:
 true_label                  1
pred_label                   2
true_class_name       versicolor
prob_of_true_class      0.261827
loss                    1.340072
correct                    False
Name: 10, dtype: object
```

5 Results

Table 1: Lab 8 result summary

Task	**Metric**	**Value**
Data split	train / val / test samples	90 / 30 / 30
Generative vs. discriminative	validation accuracy (NB / LogReg)	0.933 / 0.933 (tie)
Decision tree capacity	underfit at depth=1, overfit at depth=None	train 0.667–1.000, val 0.667–0.933
Hyperparameter selection	chosen `max_depth` / test accuracy	5 / 0.933
Cross-entropy	validation log loss	0.215
MLE vs. MAP	validation accuracy across $C \in [10^{-3}, 10^{3}]$	0.800 – 0.933

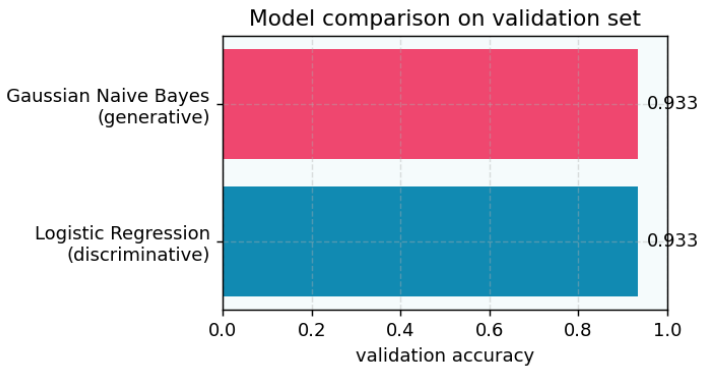

Figure 1: Validation accuracy: Logistic Regression vs. Gaussian Naive Bayes.

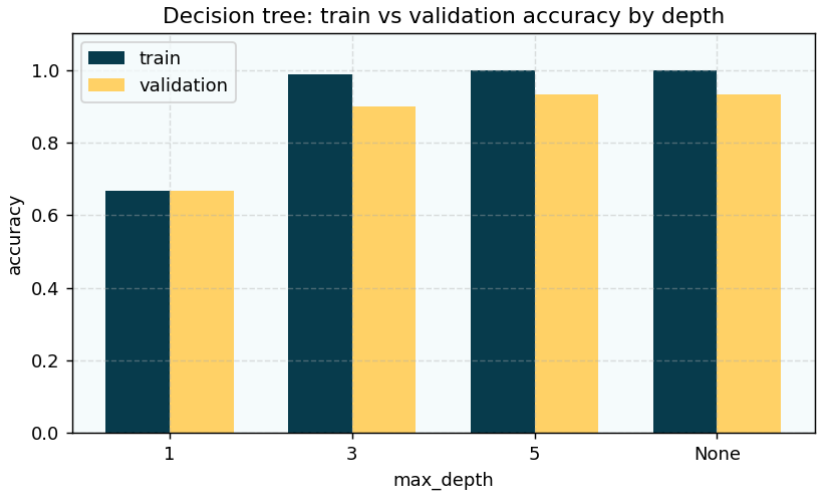

Figure 2: Training vs. validation accuracy at each Decision Tree depth.

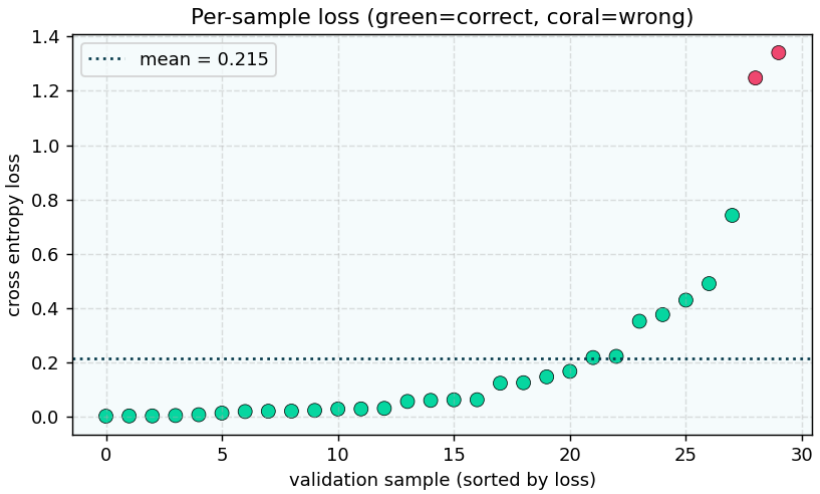

Figure 3: Per-sample cross-entropy loss on the validation set (green = correct, coral = misclassified).

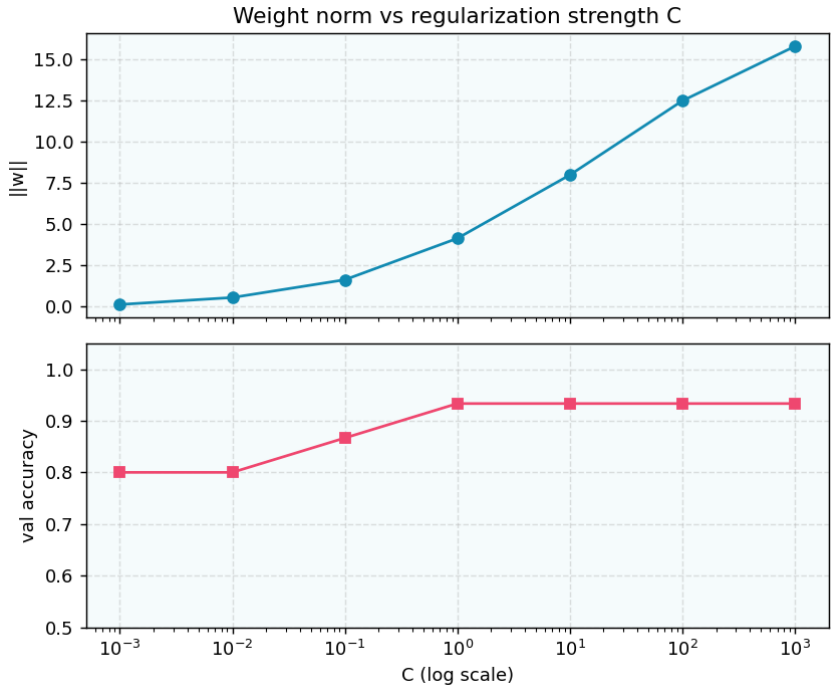

Figure 4: Weight norm and validation accuracy as the regularization strength C is swept.

6 Discussion

Gaussian Naive Bayes and Logistic Regression reach the same validation accuracy on this split. This is a reasonable outcome for Iris: its three classes are close to linearly separable and roughly Gaussian within each class, a setting that suits a linear discriminative boundary and a Gaussian generative model equally well. Naive Bayes' independence assumption between features is not strictly true here, but on a dataset this clean it costs nothing visible in the validation accuracy.

The Decision Tree capacity sweep shows the underfitting/overfitting trade-off directly. At `max_depth`=1 both training and validation accuracy are low (0.667), which is bias – the model is too simple regardless of which sample it is shown. At `max_depth`=None training accuracy reaches 1.000 while validation accuracy stays at 0.933, which is variance – the tree fits its particular training sample more closely than it fits the underlying pattern. Since `max_depth`=5 reaches the same validation accuracy with far less complexity, it was selected as the final hyperparameter and confirmed once on the test set, giving 0.933 test accuracy – close to the validation estimate that guided the choice.

The cross-entropy breakdown shows the same idea in the loss function itself: the confident correct prediction (setosa, $\hat{p} \approx 0.997$) contributes almost nothing to the loss, while the confident wrong prediction (true versicolor, predicted virginica, $\hat{p} \approx 0.262$) contributes about 390 times more, since the loss term $-\log(\hat{p})$ diverges as $\hat{p} \to 0$. Sweeping the regularization strength C makes the MLE/MAP trade-off concrete: at small C (a strong prior, MAP-like) the weights are pulled toward zero and validation accuracy drops to about 0.80; at large C (a weak prior, close to plain MLE) the weights grow and accuracy settles at 0.933. The following points summarize the lab sheet's conceptual questions:

- High training accuracy with lower validation accuracy signals overfitting – the model fit patterns specific to the training sample that do not generalize.
- A validation set lets hyperparameters be compared without spending the test set, so the final test accuracy remains an honest, unbiased estimate.
- Underfitting, high bias and low capacity go together: a simple model (`max_depth`=1) cannot represent the true boundary, so both training and validation accuracy stay low.
- Overfitting, high variance and high capacity go together: a complex model (`max_depth`=None) fits the training sample almost exactly but is sensitive to which sample it saw, so validation accuracy falls behind training accuracy.
- MAP uses a prior $p(\theta)$ over the parameters in addition to the data likelihood that MLE alone uses; MLE is MAP with a flat, uninformative prior.
- Cross-entropy penalizes a confident wrong prediction heavily because its loss is $-\log(\hat{p}_y)$, which grows without bound as $\hat{p}_y \to 0$.

7 Conclusion

This lab worked through a complete machine learning pipeline on the Iris dataset: a stratified train/validation/test split, a fair comparison between a generative and a discriminative model, a Decision Tree capacity sweep that produced textbook underfitting and overfitting, a validation-driven hyperparameter choice confirmed once on the test set, and a cross-entropy and MLE/MAP analysis of the Logistic Regression model. Every stage pointed to the same underlying lesson: the training set alone cannot tell a model or a hyperparameter choice how well it generalizes, and only held-out data – validation during development, test at the very end – can answer that honestly.